

Ergebnisbericht: Übungsblatt 3

Betriebssysteme – Wintersemester 2025/26

Universität Trier

1. Einleitung

Der vorliegende Bericht dokumentiert die Bearbeitung des dritten Übungsblatts der Vorlesung Betriebssysteme. Im Mittelpunkt steht das Dateisystem ZFS (Zettabyte File System), das sich durch Funktionen wie Copy-on-Write-Semantik, Snapshots und RAID-Unterstützung von traditionellen Dateisystemen wie ext4 unterscheidet. Die vier Aufgaben umfassen die Implementierung eines Sicherungsprogramms auf Basis von ZFS-Snapshots, die experimentelle Untersuchung von Datei-Inkonsistenzen, einen RAID-Z-Ausfalltest sowie einen vergleichenden Performance-Benchmark gegen ext4.

Als Programmiersprache wurde Python gewählt, da es über das subprocess-Modul eine direkte und gut lesbare Ansteuerung der ZFS-Kommandozeilenwerkzeuge ermöglicht. Die Testumgebung bestand aus einer Ubuntu Server 24.04 LTS VM in VirtualBox mit sechs virtuellen Festplatten (1× 20 GB System, 5× 1 GB für ZFS-Pools).

2. Aufgabe 1: Datensicherung und -archivierung mit ZFS

2.1 Konzept und Entwurfsentscheidungen

Das Backup-Programm nutzt die Copy-on-Write-Technik von ZFS-Snapshots, um einen möglichst konsistenten Zustand des zu sichernden Verzeichnisses zu erreichen. Ein Snapshot friert den Zustand eines Datasets auf Blockebene ein, ohne die Daten physisch zu kopieren. Erst bei nachfolgenden Änderungen werden die alten Blöcke beibehalten (Copy-on-Write).

Der Sicherungsablauf gliedert sich in drei Phasen: Zunächst wird ein Snapshot des Quell-Datasets erstellt (`zfs snapshot -r`). Anschließend wird der Snapshot mittels `zfs send` in eine Datei am Zielort serialisiert. Bei Vorhandensein eines vorherigen Snapshots wird `zfs send -i` verwendet, um nur die Differenz (inkrementelles Backup) zu übertragen. Abschließend prüft die Retention-Logik, ob die konfigurierte Maximalanzahl an Sicherungen überschritten wurde und entfernt gegebenenfalls die ältesten Snapshots und Backup-Dateien nach dem FIFO-Prinzip.

2.2 Kernimplementierung

Die zentrale Funktion erstellt den Snapshot und leitet den Datenstrom in eine Datei:

```
def send_snapshot_to_file(snapshot, dest_file, base_snapshot=None):
    cmd = ["zfs", "send"]
    if base_snapshot:
        cmd.extend(["-i", base_snapshot]) # inkrementell
    cmd.append(snapshot)
    with open(dest_file, "wb") as f:
        subprocess.run(cmd, stdout=f, check=True)
```

Listing 1: Snapshot-Serialisierung

2.3 Testergebnis

Die folgende Terminal-Ausgabe zeigt das Ergebnis nach fünf Backup-Läufen mit `--max-backups 3`. Die Retention-Logik hat die ältesten Sicherungen korrekt entfernt:

```
mim@zfs-system:~$ zfs list -t snapshot
NAME                                     USED  AVAIL  REFER  MOUNTPOINT
mypool/daten@backup_20260321_163915      0B    -    50K    -
mypool/daten@backup_20260321_163920      0B    -    50K    -
mypool/daten@backup_20260321_163924      0B    -    50K    -

mim@zfs-system:~$ ls -lh /mnt/backups/
total 12K
-rw-r--r-- 1 root root 624 Mar 21 16:39 backup_20260321_163915_incr.zfs
-rw-r--r-- 1 root root 624 Mar 21 16:39 backup_20260321_163920_incr.zfs
-rw-r--r-- 1 root root 624 Mar 21 16:39 backup_20260321_163924_incr.zfs
```

Abbildung 1: Retention-Ergebnis – exakt 3 Snapshots und 3 Backup-Dateien verblieben

Es verblieben exakt drei Snapshots und drei Backup-Dateien. Die inkrementellen Sicherungen (Suffix `_incr.zfs`) benötigen mit jeweils 624 Bytes deutlich weniger Speicher als das initiale Vollbackup (113 KB), da nur die Änderungen seit dem vorherigen Snapshot übertragen werden.

3. Aufgabe 2: Datei-Inkonsistenzen trotz ZFS-Snapshots

3.1 Hypothese und Experimentdesign

ZFS-Snapshots garantieren Konsistenz auf Blockebene: Kein Block wird jemals nur teilweise geschrieben (dank Copy-on-Write und transaktionalem Schreibmodell). Sie garantieren jedoch keine Konsistenz auf Anwendungsebene. Wenn eine Applikation Daten in einem internen Puffer hält und diese noch nicht per `fsync()` auf die Platte geschrieben hat, erfasst der Snapshot nur den zuletzt persistierten Zustand und nicht den vom Benutzer erwarteten aktuellen Stand.

Das Experiment verwendet einen Writer-Thread, der mit definierten Pausen zehn nummerierte Blöcke in eine Datei schreibt. Nach der Hälfte der geschätzten Schreibdauer wird ein Snapshot erstellt. Anschließend wird die Datei im Snapshot mit der fertigen Originaldatei verglichen.

3.2 Kernimplementierung

```
def slow_writer(filepath, num_blocks, delay, ...):
    with open(filepath, "w") as f:
        for i in range(1, num_blocks + 1):
            f.write(f"Block {i}: {'X' * 60}\n")
            # Kein flush/fsync -> Daten bleiben im Puffer
            time.sleep(delay)
        f.flush()
        os.fsync(f.fileno()) # erst am Ende persistiert
```

Listing 2: Writer-Thread ohne Zwischensynchronisation

3.3 Ergebnisse

Die Ausgabe des Experiments zeigt die Inkonsistenz deutlich:

```
=====
EXPERIMENT 1: Inkonsistenz durch gleichzeitiges Schreiben
=====
1. Starte Writer-Prozess (10 Blöcke, 0.5s Verzögerung)...
   Writer: Block 1/10 geschrieben
2. Warte 2.5s (bis ca. Hälfte geschrieben)...
   Writer: Block 2/10 geschrieben
   Writer: Block 3/10 geschrieben
   Writer: Block 4/10 geschrieben
   Writer: Block 5/10 geschrieben
3. Erstelle Snapshot WÄHREND des Schreibvorgangs...
   Snapshot erstellt: mypool/daten@inkonsistenz_test
4. Warte bis Writer fertig ist...
   Writer: Block 10/10 geschrieben

5. Vergleiche Original mit Snapshot:
-----
```

```
Original: 10 Blöcke (vollständig)
Snapshot: 0 Blöcke -> INKONSISTENT! Es fehlen 10 Blöcke
```

Abbildung 2: Inkonsistenz-Experiment – Snapshot enthält 0 von 10 Blöcken

Die Snapshot-Datei war vollständig leer (0 Blöcke). Da der Writer keinen Zwischenaufwurf von `flush()/fsync()` durchführte, befanden sich sämtliche Daten zum Zeitpunkt des Snapshots ausschließlich im Kernel-Schreibpuffer. Im zweiten Experiment wurde jeder Block einzeln mit `fsync()` persistiert, was zu einer vollständig konsistenten Snapshot-Kopie führte.

3.4 Lösungsansätze

Im zweiten Experiment haben wir gezeigt, dass sich die Inkonsistenz durch den gezielten Einsatz von `fsync()` und `sync` beheben lässt. Der Systemaufruf `fsync(fd)` erzwingt, dass alle im Kernel-Puffer befindlichen Daten eines bestimmten Dateideskriptors auf die Festplatte geschrieben werden. In unserem Lösungsansatz wurde nach jedem geschriebenen Block ein `f.flush()` gefolgt von `os.fsync(f.fileno())` aufgerufen. Zusätzlich wurde unmittelbar vor der Snapshot-Erstellung ein globales `sync` ausgeführt, das alle noch ausstehenden Schreibvorgänge des gesamten Systems auf die Platte erzwingt. Das Ergebnis war eine vollständig konsistente Snapshot-Kopie mit allen 10 Blöcken.

Zu beachten ist, dass `fsync()` zwar die Persistenz der Daten auf Dateisystemebene sicherstellt, jedoch keine logische Anwendungskonsistenz garantiert. Wenn eine Anwendung beispielsweise zwei zusammengehörige Dateien aktualisiert und der Snapshot zwischen den beiden Schreibvorgängen erstellt wird, kann der Snapshot trotzdem einen inkonsistenten Anwendungszustand enthalten. Für solche Fälle wären weitergehende Maßnahmen wie Anwendungscoordination (z.B. Write-Ahead-Logging bei Datenbanken) oder das Einfrieren des Dateisystems mittels `fsfreeze` erforderlich.

4. Aufgabe 3: Datensicherheit mit RAID-Z

4.1 Experimentaufbau

Es wurde ein RAID-Z1-Pool aus drei virtuellen Festplatten erstellt (`/dev/sdc`, `/dev/sdd`, `/dev/sde`). RAID-Z1 verteilt Daten und eine Paritätsinformation über alle Platten, sodass der Ausfall einer beliebigen Platte toleriert werden kann.

```
zpool create raidpool raidz /dev/sdc /dev/sdd /dev/sde
zfs create raidpool/daten
# Plattenausfall simulieren:
zpool offline raidpool /dev/sdc
# Wiederherstellung:
zpool online raidpool /dev/sdc # -> automatisches Resilvering
```

Listing 3: RAID-Z Pool-Erstellung und Ausfallsimulation

4.2 Ergebnisse

Der Pool-Status nach dem simulierten Plattenausfall:

```
pool: raidpool
state: DEGRADED
status: One or more devices has been taken offline by the
        administrator.
config:

    NAME        STATE        READ WRITE CKSUM
    raidpool    DEGRADED    0     0     0
      raidz1-0  DEGRADED    0     0     0
        sdc     OFFLINE    0     0     0
        sdd     ONLINE    0     0     0
        sde     ONLINE    0     0     0

errors: No known data errors
```

Abbildung 3: Pool-Status nach Ausfall von `/dev/sdc`

Die IO-Operationen und Datenintegritätsprüfung während des Ausfalls:

```
--- IO-Operationen nach Plattenausfall prüfen ---
Operationen nach Ausfall: 15
Fehler nach Ausfall: 0
-> ERFOLGREICH: Alle Lese-/Schreiboperationen liefen
    ohne Unterbrechung!

--- Datenintegrität prüfen (Prüfsummen vergleichen) ---
wichtige_daten_1.txt: OK
wichtige_daten_2.txt: OK
wichtige_daten_3.txt: OK

--- Pool-Status nach Recovery ---
scan: resilvered 28K in 00:00:00 with 0 errors
Pool-Zustand nach Recovery: ONLINE
```

Abbildung 4: IO-Operationen und Datenintegrität nach Plattenausfall

Messgröße	Ergebnis
Pool-Zustand nach Ausfall	DEGRADED (funktionsfähig)
IO-Operationen nach Ausfall	15 (alle erfolgreich)
IO-Fehler während Ausfall	0
Datenintegrität (SHA256)	Alle 3 Dateien identisch
Recovery-Dauer (Resilvering)	< 1 Sekunde
Pool-Zustand nach Recovery	ONLINE

Tabelle 2: Zusammenfassung des RAID-Z Ausfall experiments

5. Aufgabe 4: Zugriffssperformanz ZFS vs. ext4

5.1 Benchmark-Methodik

Der Benchmark vergleicht ZFS und ext4 auf jeweils einer separaten virtuellen Festplatte (je 1 GB) in derselben VM. Jeder Test wurde dreimal durchgeführt, wobei vor jeder Messung der Kernel-Cache geleert wurde (`echo 3 > /proc/sys/vm/drop_caches`), um Verfälschungen durch Caching-Effekte zu minimieren.

5.2 Ergebnisse

Die vollständige Benchmark-Ausgabe:

```
=== Test 1: Sequentielles Schreiben ===
ZFS:   Durchschnitt: 183.9 MB/s
EXT4:  Durchschnitt: 218.8 MB/s

=== Test 2: Sequentielles Lesen ===
ZFS:   Durchschnitt: 769.9 MB/s
EXT4:  Durchschnitt: 282.7 MB/s

=== Test 3: Zufälliges Schreiben ===
ZFS:   Durchschnitt: 440 ops/s
EXT4:  Durchschnitt: 391 ops/s

=== Test 4: Zufälliges Lesen ===
ZFS:   Durchschnitt: 28806 ops/s
EXT4:  Durchschnitt: 2680 ops/s

=== Test 5: Metadaten-Performance ===
ZFS:   Erstellen 12015 ops/s, Löschen 12935 ops/s
EXT4:  Erstellen 17731 ops/s, Löschen 45061 ops/s
```

Abbildung 5: Zusammenfassung der Benchmark-Ergebnisse

Test	ZFS	ext4	Faktor
Seq. Schreiben	183,9 MB/s	218,8 MB/s	ext4 1,2×
Seq. Lesen	769,9 MB/s	282,7 MB/s	ZFS 2,7×
Random Schreiben	440 ops/s	391 ops/s	ZFS 1,1×
Random Lesen	28.806 ops/s	2.680 ops/s	ZFS 10,7×
Metadaten Erstellen	12.015 ops/s	17.731 ops/s	ext4 1,5×
Metadaten Löschen	12.935 ops/s	45.061 ops/s	ext4 3,5×

Tabelle 3: Performance-Vergleich ZFS vs. ext4 (Durchschnitt aus 3 Läufen)

5.3 MB/s vs. ops/s

Die Benchmark-Ergebnisse verwenden zwei unterschiedliche Metriken, die jeweils das Zugriffsmuster widerspiegeln. Bei den sequentiellen Tests (Lesen/Schreiben einer großen 256-MB-Datei) ist der Durchsatz in **MB/s** die aussagekräftige Größe, da hier das Volumen der übertragenen Daten pro Zeiteinheit entscheidend ist. Bei den Random- und Metadaten-Tests hingegen werden viele kleine, voneinander unabhängige Operationen durchgeführt (500 Dateien à 4 KB bzw. 1000 leere Dateien erstellen/löschen). Hier ist nicht das Datenvolumen der Bottleneck, sondern die Anzahl der Operationen, die das Dateisystem pro Sekunde, gemessen in **ops/s** (operations per second), abarbeiten kann. Diese Metrik bildet Szenarien wie Webserver, Datenbanken oder Paketmanager ab, bei denen viele kleine Dateizugriffe die Last dominieren.

5.4 Analyse

Sequentielles Schreiben (ext4 +19%): Der Overhead von ZFS durch Copy-on-Write, Prüfsummenberechnung (SHA-256 für jeden Block) und transaktionales Schreiben führt zu einer messbaren Verlangsamung. Jeder Schreibvorgang erfordert bei ZFS zusätzliche Metadaten-Updates im Merkle-Baum, während ext4 Daten direkt in vorab allozierte Extents schreiben kann.

Sequentielles Lesen (ZFS 2,7×): ZFS profitiert hier erheblich von seinem ARC (Adaptive Replacement Cache), der intelligenter als der Linux Page Cache arbeitet und sowohl Häufigkeit als auch Aktualität von Zugriffen berücksichtigt.

Random Lesen (ZFS 10,7×): Der mit Abstand größte Unterschied zeigt sich beim zufälligen Lesen kleiner Dateien. Der ARC-Cache von ZFS behält häufig zugriffene Metadaten und Datenblöcke vor und vermeidet so teure Disk-Seekes.

Metadaten-Operationen (ext4 1,5–3,5×): ext4 ist bei reinen Metadaten-Operationen deutlich schneller, was auf die schlankere Datenstruktur zurückzuführen ist. ZFS verwaltet für jede Datei zusätzliche Metadaten (Prüfsummen, erweiterte Attribute, Transaktionsgruppen).

6. Fazit

Die durchgeführten Experimente demonstrieren die Stärken und Grenzen von ZFS als modernes Dateisystem. Die Copy-on-Write-Semantik ermöglicht effiziente und nahezu instantane Snapshots für Backup-Zwecke (Aufgabe 1), garantiert jedoch keine Anwendungskonsistenz ohne zusätzliche Maßnahmen (Aufgabe 2). Die integrierte RAID-Z-Unterstützung bietet robusten Ausfallschutz mit transparentem Failover für laufende Anwendungen (Aufgabe 3). Im Performance-Vergleich zeigt sich ZFS als überlegen bei Leseoperationen dank seines ARC-Caches, während ext4 bei Schreib- und Metadatenoperationen effizienter arbeitet (Aufgabe 4).